Recommender Systems

Collaborative and content-based filtering

The problem

- Users × items rating matrix, mostly empty
- Predict the missing ratings
- Recommend the highest predictions

Matrix factorisation

$$R \approx PQ^\top, \quad \hat{r}_{ui} = p_u \cdot q_i$$

- p_u: user taste vector (k dims)
- q_i: item trait vector (k dims)
- Fit only observed entries

Alternating least squares

$$p_u = (Q_o{}^\top Q_o + \lambda I)^{-1} Q_o{}^\top r_u$$

- Fix Q → each user is a small ridge regression
- Fix P → each item is a ridge regression
- Alternate until it settles

ALS

```
def fit(self, R, mask=None):
    R = np.asarray(R, float)
    M = ~np.isnan(R) if mask is None else np.asarray(mask, bool)
    R = np.nan_to_num(R)
    rng = np.random.default_rng(self.seed)
    n, m = R.shape
    P, Q = rng.normal(0, 0.1, (n, self.k)), rng.normal(0, 0.1, (m, self.k))
    I = self.reg * np.eye(self.k)
    for _ in range(self.iters):
        for u in range(n):
            Qo = Q[M[u]]
            P[u] = np.linalg.solve(Qo.T @ Qo + I, Qo.T @ R[u, M[u]])
        for i in range(m):
            Po = P[M[:, i]]
            Q[i] = np.linalg.solve(Po.T @ Po + I, Po.T @ R[M[:, i], i])
    self.P, self.Q = P, Q
    return self
```

Content-based filtering

- Describe items by features (genre, tags, text)
- Cosine similarity between item vectors
- Recommend items similar to what the user liked
- Solves item cold-start

Cosine similarity

```
def content_similarity(item_features):
    F = np.asarray(item_features, float)
    norms = np.linalg.norm(F, axis=1, keepdims=True)
    F = F / np.where(norms == 0, 1, norms)
    return F @ F.T
```

Trade-offs

- ALS: $O(\text{iters} \cdot (n + m) \cdot (k^3 + \text{nnz} \cdot k))$
- CF needs interaction history (cold start)
- Content-based: limited novelty
- Production: hybrid + two-stage retrieval → ranking

Recap

- Ratings ≈ user · item factors
- ALS = alternating ridge regressions
- Content similarity for cold start
- Combine both in practice